

Bootcamp: Full Stack Junior

Modulo 4: Profesionalización Fronted con TypeScript y Next.js

Semana 11

Contenido

1

Introducción a TypeScript

- ¿Qué es TypeScript y por qué usarlo?
- Instalación y configuración (Node.js, TS compiler, tsconfig.json)
- Tipos primitivos (string, number, boolean, null, undefined)
- Arrays, tuplas y enums
- Diferencias con JavaScript

2

Tipos avanzados y especiales

- void: funciones sin retorno
- never: funciones que nunca retornan
- unknown: alternativa segura a any

3

Interfaces y tipos para contratos de API

- Interfaces vs Type aliases
- Propiedades opcionales y readonly

4

Consumo de servicios y PNPM

Contenido:

1. ¿Qué es TypeScript y por qué usarlo?

TypeScript (TS) es un **superset de JavaScript**, lo que significa que **todo código JavaScript válido también es TypeScript**, pero TypeScript agrega una capa extra de **tipado estático** y herramientas de desarrollo avanzadas.

En **JavaScript**, el tipado es **dinámico**, lo que implica que las variables pueden cambiar de tipo en tiempo de ejecución. Esto es flexible, pero en proyectos grandes suele generar errores difíciles de detectar.

TypeScript introduce el **tipado estático**, permitiendo detectar errores **antes de ejecutar el código**, directamente en el editor o durante la compilación. Esto es clave para:

- Proyectos escalables
- Trabajo en equipo
- Código mantenible y profesional

En entornos como **Next.js**, TypeScript no es solo una opción: es un **estándar de la industria**.

Característica	JavaScript (JS)	TypeScript (TS)
Tipado	Dinámico	Estático (opcional pero fuerte)
Detección de errores	En ejecución	En desarrollo / compilación
Autocompletado	Limitado	Avanzado e inteligente
Escalabilidad	Media	Alta
Mantenibilidad	Difícil en proyectos grandes	Ala
Uso en frameworks modernos	Compatible	Recomendado / preferido

Tabla 1 - JavaScript vs TypeScript - Fuente: Autor de Contenidos

1.1. Instalación y Configuración

Preparación del Entorno

TypeScript se instala como una dependencia de desarrollo dentro de proyectos modernos basados en Node.js.

```
npm init -y
npm install typescript --save-dev
npx tsc --init
```

Ilustración 1 - Inicializar TypeScript - Fuente: Autor de contenidos

Estos comandos:

1. Inician el proyecto
2. Instalan TypeScript
3. Generan el archivo de configuración `tsconfig.json`

1.2. ¿Qué es `tsconfig.json`?

Es el **archivo central de configuración** de TypeScript. Define cómo se analiza, valida y transpila el código.

Opción	Descripción
target	Versión de JavaScript de salida (ej: ES2017, ES2020)
module	Sistema de módulos (ESNext, CommonJS)
strict	Activa validaciones estrictas de tipos
outDir	Carpeta donde se genera el JS compilado

Tabla 2 - Partes de un archivo `tsconfig.json` - Fuente: Autor de Contenidos

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "strict": true,
    "outDir": "dist"
  }
}
```

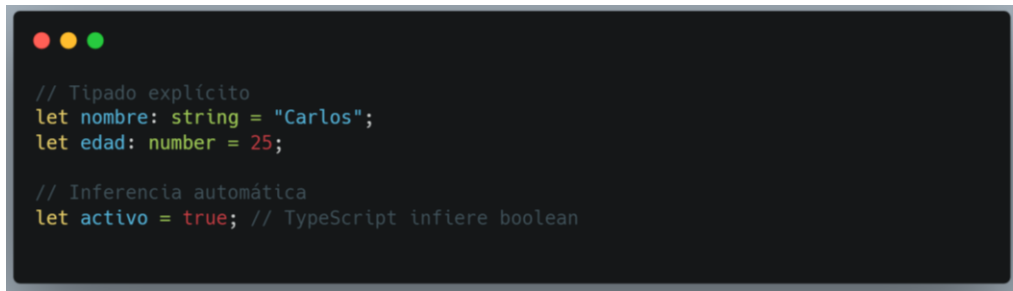
Ilustración 2 - Ejemplo de un archivo `tsconfig` - Fuente: Autor de contenidos

1.3. Tipos Primitivos

TypeScript define tipos primitivos para representar valores básicos, permitiendo mayor claridad y control en el código.

Los tipos primitivos más usados son:

- string
- number
- boolean
- null
- undefined



```
// Tipado explícito
let nombre: string = "Carlos";
let edad: number = 25;

// Inferencia automática
let activo = true; // TypeScript infiere boolean
```

Ilustración 3 - Tipado Explícito vs Inferencia - Fuente: Autor de contenidos

TypeScript **recomienda la inferencia** cuando el tipo es claro, reservando el tipado explícito para:

- Funciones
- APIs
- Modelos de datos

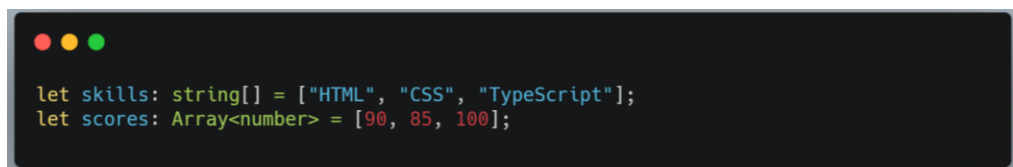
Tipo	Ejemplo de Valor	Uso Común
string	"Hola"	Textos, nombres, IDs
number	42	Cálculos, edades
boolean	true	Flags, estados
null	null	Valores intencionalmente vacíos
undefined	undefined	Valores no inicializados

Tabla 3 - Tipos Primitivos - Fuente: Autor de Contenidos

1.4. Arrays, Tuplas y Enums

Arrays

Colecciones de elementos del **mismo tipo**.



```
let skills: string[] = ["HTML", "CSS", "TypeScript"];
let scores: Array<number> = [90, 85, 100];
```

Ilustración 3 - Arrays en TypeScript - Fuente: Autor de contenidos

Tuplas

Arreglos de **longitud fija** y **tipos definidos por posición**.

```
let userTuple: [number, string] = [1, "Ana"];
```

Ilustración 4 – Tuplas en TypeScript - Fuente: Autor de contenidos

Son útiles para:

- Respuestas de APIs
- Pares clave-valor simples

Enums

Permiten definir un conjunto de **valores constantes y legibles**.

```
enum Rol {  
  Admin,  
  User,  
  Guest  
}
```

Ilustración 5 – Enums en TypeScript - Fuente: Autor de contenidos

Ejemplo Integrado: Usuario

```
enum Rol {  
  Admin,  
  User  
}  
  
type Usuario = {  
  id: number;  
  nombre: string;  
  habilidades: string[];  
  rol: Rol;  
};  
  
const usuario: Usuario = {  
  id: 1,  
  nombre: "Laura",  
  habilidades: ["React", "Next.js"],  
  rol: Rol.Admin  
};
```

Ilustración 6 – Ejemplo de Enums, Tuplas y Arrays - Fuente: Autor de contenidos

1.5. Diferencias Clave con JavaScript

Compilación vs Ejecución

- **TypeScript se transpila** a JavaScript
- Los errores se detectan **antes de ejecutar la app**
- JavaScript detecta errores **en tiempo de ejecución**

¿Por qué Next.js prefiere TypeScript?

1. Mejor autocompletado y DX (Developer Experience)
2. Menos bugs en producción
3. Código auto-documentado
4. Escalabilidad para equipos grandes
5. Integración nativa y oficial

2. Introducción a Tipos Especiales (void, never, unknown)

En TypeScript existen **tipos especiales** diseñados para describir **comportamientos**, no valores. Son clave para **prevenir errores antes de que lleguen al navegador** y para que React + TypeScript se lleven bien en proyectos reales.

Piensa en estos tipos como **señales de tránsito**:

- **void** dice: “esta función *no devuelve nada útil*”.
- **never** dice: “este camino *nunca termina o siempre falla*”.
- **unknown** dice: “no sé qué es esto *todavía*, pero lo verificaré antes de usarlo”.

2.1. void: cuando una función no retorna valor

En React, muchos handlers (como onClick) **no necesitan devolver nada**. El tipo void deja esto claro y evita suposiciones incorrectas.



```
// Handler típico en React
const handleClick = (): void => {
  console.log("Botón presionado");
};
```

Usar void comunica intención: *esta función existe por su efecto, no por su retorno.*

2.2. never: cuando algo nunca ocurre

never se usa cuando una función **no puede terminar normalmente**:

- Lanza un error siempre
- Entra en un bucle infinito

Esto ayuda a TypeScript a detectar **código inalcanzable**.



```
function throwError(message: string): never {  
  throw new Error(message);  
}
```

Ilustración 8 – Ejemplo de never- Fuente: Autor de contenidos

Aquí TypeScript sabe que **nada después de esta llamada se ejecuta**, lo que mejora la seguridad del flujo.

2.3. unknown: la alternativa segura a any

unknown representa un valor que **no conocemos aún**, pero obliga a **validarlo antes de usarlo**. Es perfecto para:

- Respuestas de APIs
- Datos externos
- Inputs dinámicos



```
let data: unknown;  
  
data = "Hola";  
  
// Error: no se puede usar directamente  
// console.log(data.toUpperCase());  
  
// Validación obligatoria  
if (typeof data === "string") {  
  console.log(data.toUpperCase());  
}
```

Ilustración 9 – Ejemplo de unknow- Fuente: Autor de contenidos

Con unknown, **TypeScript te obliga a pensar**, y eso previene errores reales.

2.4. Tabla Comparativa: any vs unknown vs never

Tipo	¿Permite cualquier operación?	¿Requiere validación?	Uso recomendado
any	Sí	No	Evitar
unknown	No	Si	Recomendado
never	No	No aplica	Casos extremos

Tabla 4 - any vs unknown vs never - Fuente: Autor de Contenidos

3. La Base de los Contratos – Interfaces vs Type Aliases

Cuando trabajamos con **APIs**, necesitamos definir **contratos claros**: estructuras que describan **exactamente** qué datos esperamos recibir o enviar. En TypeScript, estos contratos se definen principalmente con **Interfaces** y **Type Aliases**.

Una **Interface** es como un **contrato formal**: define la forma que un objeto *debe cumplir*. Es ideal para:

- Modelar objetos complejos
- Representar entidades del backend (User, Post, Product)
- Trabajar con props y estados en React

Un **Type Alias**, en cambio, es una **etiqueta** para un tipo. Es más flexible y se usa mejor para:

- Uniones (|)
- Intersecciones (&)
- Tipos primitivos combinados

Característica	Interface	Type Alias
Declaración	interface User {}	type User = {}
Extensión	extends	&
Reapertura	Si	No
Ideal para	Contratos de objetos	Estados, uniones
Uso en APIs	Muy usado	Poco común de usar

Tabla 5 - Comparativa Técnica - Fuente: Autor de Contenidos

Implementación en Código

```
// Contrato de datos de usuario (API)
interface User {
  id: number;
  name: string;
  email: string;
}

// Estado o condición del usuario
type UserStatus = "active" | "inactive" | "banned";
```

Ilustración 10 – Interfaces y Types en TypeScript- Fuente: Autor de contenidos

3.1. Modificadores de Propiedades (Optional & Readonly)

as APIs **no siempre envían todos los campos** y los datos recibidos **no deberían modificarse** directamente. TypeScript nos ayuda con dos modificadores clave:

- **Optional (?)** → El dato *puede o no existir*
- **Readonly** → El dato *no debe cambiarse*

Estos modificadores refuerzan la idea de que el frontend **consume datos**, no los muta arbitrariamente.

Implementación en Código

```
interface Post {
  readonly id: number;           // Nunca debe cambiar
  title: string;
  content: string;
  createdAt: string;
  updatedAt?: string;           // Puede no venir del backend
}
```

Ilustración 11 – Interface readonly y optional- Fuente: Autor de contenidos

```
const post: Post = {
  id: 1,
  title: "Hola TypeScript",
  content: "Contenido del post",
  createdAt: "2026-01-01"
};

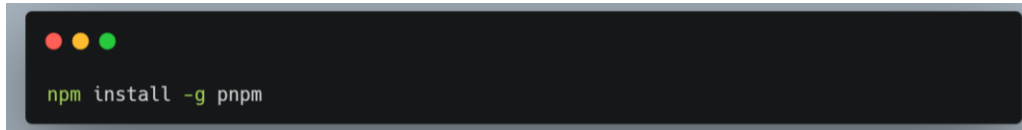
// ✖ Error de TypeScript
// post.id = 2;
```

Ilustración 12 – Implementación de Interface- Fuente: Autor de contenidos

4. PNPM: El Gestor de Paquetes Eficiente

PNPM es un gestor de paquetes moderno para JavaScript que prioriza eficiencia, velocidad y ahorro de espacio en disco. A diferencia de NPM o Yarn, PNPM utiliza un enfoque llamado content-addressable storage.

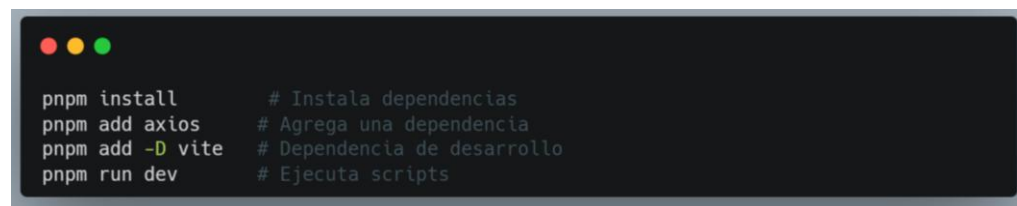
Instalación Global



```
npm install -g pnpm
```

Ilustración 13 –Implementación global pnpm - Fuente: Autor de contenidos

Comandos Esenciales



```
pnpm install      # Instala dependencias
pnpm add axios    # Agrega una dependencia
pnpm add -D vite  # Dependencia de desarrollo
pnpm run dev      # Ejecuta scripts
```

Ilustración 13 –Opciones de pnpm - Fuente: Autor de contenidos

4.1. Tabla Comparativa: NPM vs PNPM vs YARN

Característica	NPM	YARN	PNPM
Velocidad	Media	Alta	Muy alta
Uso de disco	Alto	Medio	Muy bajo
node_modules	Duplicado	Semi optimizado	Enlaces simbólicos
Ideal para	Proyectos simples	Equipos	Proyectos grandes

Tabla 6 - NPM vs PNPM vs YARN - Fuente: Autor de Contenidos

Recursos de Apoyo



Video

[Curso TypeScript desde cero \(1 hora, configuración, tipos básicos\)](#)



Video

[Tutorial de programacion orientada a objetos con TypeScript](#)



Guia

[Guía de uso de pnpm](#)